46. Node Metrics and Exporters (ops intro)

46. Node Metrics and Exporters (ops intro)

Phase: Phase 10 — Observability and troubleshooting
Estimated time: 1–2 days
Slug: `46-prometheus-node`

Learning objectives

- Expose node metrics conceptually
- Know RED/USE for hosts
- Alert on disk/inode/memory
- Avoid high-cardinality labels

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

You cannot operate what you cannot see. Most Linux outages are slow ones: a disk that fills over three days, memory that creeps, inodes that run out on a log directory. Humans do not notice these; a metrics stack does. The common open-source setup is `node_exporter` on each host, Prometheus scraping it, Grafana drawing graphs, and Alertmanager paging someone. Knowing what to alert on, and just as important what not to, keeps your pager useful.

A classic first alert is disk usage. A common starting threshold is 90 percent used on a data filesystem, with a lower warning earlier. At 90 you still have time to act; at 100 databases stop and logs fail.

Core concepts

Idea	Meaning
Exporter	process that exposes metrics over HTTP (`:9100/metrics`)
Scrape	Prometheus pulls metrics on an interval
Counter	only goes up (`node_network_receive_bytes_total`)
Gauge	goes up and down (`node_memory_MemAvailable_bytes`)
Label	key/value dimension (`mountpoint="/"`)
Cardinality	number of unique label combinations; keep it low
USE method	Utilization, Saturation, Errors for resources
RED method	Rate, Errors, Duration for services

```
  host: node_exporter :9100 --metrics--+
  host: node_exporter :9100 --metrics--+--> Prometheus (scrape 15s, TSDB)
  host: node_exporter :9100 --metrics--+        |
                                                +--> Alertmanager --> pager
                                                +--> Grafana dashboards
```

Alert on symptoms users feel or resources about to run out, not on every spike. An alert should be actionable, and have a clear owner.

Worked examples

Example 1: look at raw exporter output.

```
curl -s localhost:9100/metrics | grep -E '^node_filesystem_(avail|size)_bytes' |
head
curl -s localhost:9100/metrics | grep node_load1
```

Example 2: a Prometheus scrape config.

```
scrape_configs:
  - job_name: node
    scrape_interval: 15s
    static_configs:
      - targets: ['10.0.0.11:9100', '10.0.0.12:9100']
```

Example 3: disk percent used, and the alert at 90.

```
groups:
- name: host
  rules:
  - alert: DiskAlmostFull
    expr: (1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"} /
node_filesystem_size_bytes{fstype!~"tmpfs|overlay"}) * 100 > 90
    for: 10m
    labels: {severity: page}
```

```
    annotations:
      summary: "{{ $labels.instance }} {{ $labels.mountpoint }} over 90 percent"
```

The `for: 10m` prevents flapping. Excluding tmpfs and overlay avoids noise.

Example 4: inodes and memory.

```
node_filesystem_files_free / node_filesystem_files < 0.10
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes < 0.10
```

Example 5: predict instead of react.

```
predict_linear(node_filesystem_avail_bytes{mountpoint="/"}[6h], 24*3600) < 0
```

This fires when the trend says the disk will be full within a day, giving you time to fix it during working hours.

Mini-lab

1. Install `prometheus-node-exporter` (or download node_exporter) in a lab VM and start it.
2. Run `curl -s localhost:9100/metrics | wc -l` and find three metrics you recognise from `free`, `df`, and `uptime`.
3. Run `df -h /` and compute the percent used by hand from the exporter's avail and size values.
4. Fill a scratch loopback filesystem or use `fallocate -l 1G /tmp/big` and watch the metric change after the next scrape.
5. Write an alert rule file with the 90 percent rule and validate it: `promtool check rules rules.yml`.
6. List one label you must never put on a metric, such as a user id or request id, and say why.

Decision guide / failure modes

Question	Choice
Page a human?	only if user impact is happening or imminent
Ticket instead?	slow trends, 80 percent disk, cert in 14 days
Which method?	USE for hosts and disks, RED for services
Threshold or prediction?	prediction for fills, threshold for hard caps
New label?	only if cardinality is bounded and small

Failure modes: alerting on CPU above 80 percent with no context; no `for:` clause, so every blip pages; high-cardinality labels such as `user_id` that blow up the TSDB; scraping only the app and forgetting the host; and alerts without a runbook link. Test alert rules with `promtool` and trigger them once on purpose so you know the route to the pager works.

Common mistakes

- Paging on noisy low-value alerts
- No runbooks
- Cardinality explosions

Exercises

Work in `exercises/46-prometheus-node/`. Read that folder's `README.md` fully before coding.

```
cd exercises/46-prometheus-node
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `alert_on_disk_pct`. It prints the numeric percent-used threshold that the lesson uses as the example disk alert, as a bare integer with no percent sign and no text. No metrics stack is required for the test. After it passes, install node_exporter in a lab VM, compute the percent used on `/` from the raw `node_filesystem_*` metrics, and compare it with `df`. Write the alert expression from the lesson in your own words, and note why `for: 10m` and a tmpfs exclusion are part of a good rule. Also note one warning-level threshold that should fire earlier and go to a ticket.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/46-prometheus-node/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Name three host alerts
2. USE method?
3. Why inode alerts?

Further reading

- https://prometheus.io/docs/guides/node-exporter/